

99. Les lutins : trois nuances et la transparence

Un lutin n'a que trois nuances : l'indice 0 est un trou, par où l'on voit le décor.

1. Ce qu'il faut comprendre

Les **lutins** (les sprites) ont leurs propres palettes — **deux** sur la Game Boy d'origine — réglées par `paletteLutins(numero, n0, n1, n2, n3)`.

Mais chez eux, **l'indice 0 ne s'affiche jamais** : il est **transparent**. Quelle que soit la palette, un pixel 0 laisse voir le décor derrière. C'est ce qui permet à un personnage rond de ne pas se promener dans un carré blanc. Un lutin n'a donc que **trois** nuances visibles : 1, 2 et 3.

Le premier lutin prend la palette 0 (celle par défaut). Le second demande la palette 1 avec l'option `PALETTE1` de `sprite()`. **Le même dessin**, deux palettes : deux personnages différents pour le prix d'une tuile. Les jeux Game Boy s'en servent pour distinguer un ennemi fort d'un faible, ou le joueur 1 du joueur 2.

Le décor est tramé exprès : regarde les coins des deux pions — on y voit le décor, pas un fond uni.

À toi : fais `paletteLutins(1, 0, 1, 1, 1)` : que devient le second pion ?

Les fonctions de cette leçon — et le tuto de chacune

- `#include <Tuile>` → [0.107.2. Tuile — dessiner une case](#)
- `#include <ecran>` → [27.1. La fonction `ecran\(\)` — éteindre et rallumer l'écran](#)
- `#include <poser>` → [0.1.3. La fonction `poser\(\)` — poser une tuile](#)
- `#include <texte>` → [0.0.2. La fonction `texte\(\)` — écrire un mot](#)
- `#include <paletteLutins>` → [98.1. La fonction `paletteLutins\(\)` — les nuances des lutins](#)
- `#include <sprite>` → [0.69.4. La fonction `sprite\(\)` — un lutin au pixel près](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **584 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	57 o	← la console, à l'allumage — la mise en route de la console
main()	143 o	← démarrage — ta fonction
VBlank	8 o	← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage	28 o	← <code>image()</code> — attend l'image suivante
AttendreVBlank	16 o	← <code>ecran()</code> — attend que l'écran soit libre
AttendreAcces	12 o	← <code>EcrireTexte</code> — attend de pouvoir écrire une case
AdresseCase	10 o	← <code>poser()</code> — trouve l'adresse d'une case de l'écran
EcrireTexte	10 o	← <code>texte()</code> — écrit un texte, lettre par lettre
RangerLutins	11 o	← démarrage — range les 40 lutins hors de l'écran
InstallerTransfert	15 o	← démarrage — installe la copie des lutins en mémoire rapide
CopierTuiles	85 o	← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage — vide les deux cartes de l'écran
Tuiles	144 o	← <code>CopierTuiles</code> — 7 lettres sur 43, et 2 tuiles dessinées
DonneesTransfert	10 o	← <code>InstallerTransfert</code> — la petite routine qui copie les lutins

```
"PALETTE 0 PAL..." 21 o ← texte() – un texte gravé
Lettres gravées : AELPT01
Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EffacerCases, EcrireNombre, Diviser, Reste, Multiplier,
DecalerGauche, DecalerDroite, Hasard, EffacerFond, EffacerPanneau, LireManette, Notes
Fonctions du langage appelées :
sprite()      × 2    47 o sur place, dans main()
ecran()       × 2    21 o sur place, dans main() + AttendreVBlank (16 o)
poser()       × 1    13 o sur place, dans main() + AdresseCase (10 o)
texte()       × 1    11 o sur place, dans main() + EcrireTexte (10 o)
image()       × 1     6 o sur place, dans main() + AttendreImage (28 o)
paletteLutins() × 1     4 o sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// CE PROGRAMME : deux pions ronds posés sur un décor gris.
// Même dessin, mais deux palettes différentes : ils n'ont pas les mêmes nuances.
//
// Ce qui est nouveau : le LUTIN (en anglais « sprite »).
// Un lutin est un dessin de 8 × 8 qui se pose AU PIXEL près, par-dessus le décor.
// sprite(numero, x, y, TUILE) : numero = quel lutin (0 à 39),
//                                x, y = sa place en pixels (l'écran fait 160 × 144).
// Chez les lutins, l'indice 0 est TRANSPARENT : on voit le décor à travers.
// paletteLutins(numero de palette, n0, n1, n2, n3) : comme paletteFond,
// pour les lutins. Il y en a deux : la palette 0 et la palette 1.
```

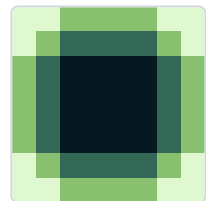
```
// Le pion : les coins en 0, donc transparents.
```

```
#include <Tuile>          // un dessin de 8 × 8 pixels
#include <ecran>          // éteint ou rallume l'écran
#include <poser>          // pose une tuile sur une case du fond
#include <texte>          // écrit un texte à l'écran
#include <paletteLutins>  // choisit les quatre nuances des lutins
#include <sprite>         // place un lutin de 8 × 8 au pixel près
```

2. UN DESSIN — PION

```
Tuile PION = {
  "00111100",
  "01222210",
  "12333321",
  "12333321",
  "12333321",
  "12333321",
  "01222210",
  "00111100",
};

// Le décor : une tuile toute en indice 2 (gris sombre).
```



à l'écran

3. UN DESSIN — DECOR

```
Tuile DECOR = {
  "22222222", "22222222", "22222222", "22222222",
  "22222222", "22222222", "22222222", "22222222",
};
```



à l'écran

4. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {                                // Le jeu commence ici.
    ecran(0);                               // Dessin écran éteint.
    for (uint8_t l = 7; l < 13; l++) {      // Les lignes 7 à 12...
        for (uint8_t c = 0; c < 20; c++) {  // ... sur toute la largeur :
            poser(c, l, DECOR);             // du décor gris.
        }
    }
    texte(1, 2, "PALETTE 0  PALETTE 1");
    ecran(1);                               // On rallume.

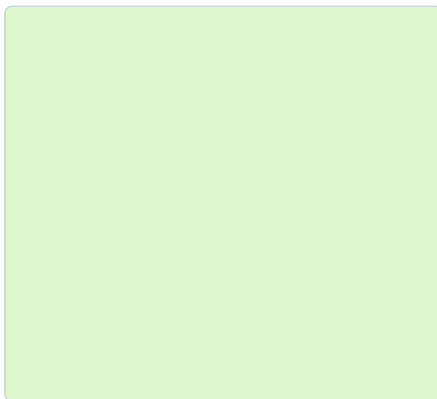
    // La palette 1 des lutins : les indices 1, 2, 3 deviennent 3, 1, 0.
    // (Le 0 du début est l'indice 0 : il reste transparent quoi qu'on mette.)
    paletteLutins(1, 0, 3, 1, 0);

    sprite(0, 40, 72, PION);                // Lutin 0, au pixel (40, 72), palette 0 : celle d'origine.
    sprite(1, 112, 72, PION, PALETTE1);    // Lutin 1 : le même dessin, palette 1.
                                           // 72 = 9 × 8 : les deux sont au milieu du décor.

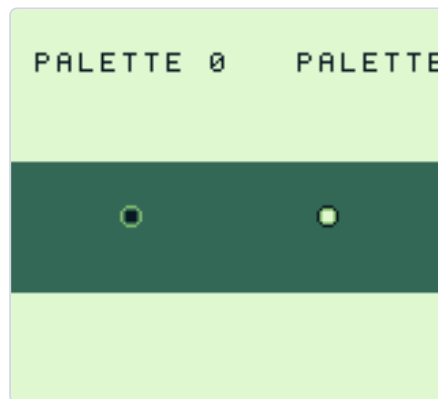
    while (true) {                          // La boucle du jeu.
        image();
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Deux pions identiques de forme, posés sur un décor gris : l'un sombre au centre, l'autre clair.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ le coin du premier pion laisse voir le décor

- ✓ le coin du second aussi
- ✓ le centre du premier : indice 3, nuance 3
- ✓ le centre du second : indice 3, nuance 0 par sa palette

Ensuite : **100. Faire clignoter un personnage touché** — chapitre 15 : Les quatre nuances : la Game Boy d'origine. · gameboy3 — écrire une cartouche Game Boy en C++